FinTwin

An AI-Assisted Banking Database Performance Simulator and Performance Digital Twin for PostgreSQL

FINAL PROJECT REVIEW REPORT

Database Systems Project

Submitted by

Team Member	Registration Number
Arav Singh Patel	24BCE2425
Chaitanya Ghogare	24BCE2392

B.Tech Computer Science and Engineering

VIT Vellore

October 2026 (revised edition)

Source repository: https://github.com/Arrav-SP/FinTwin

Abstract

Relational database performance depends on the workload that is applied: the mix of operations, the arrival rate and the number of concurrent clients. Measuring every configuration of interest directly is slow, so it is attractive to learn the relationship between workload and performance from a limited set of measured experiments and to use it to estimate the behaviour of configurations that have not been run. FinTwin is a prototype of such a system for a banking workload on PostgreSQL 16.

The project implements a complete measure, learn, predict and validate loop. A normalized banking schema of six tables is populated with deterministic synthetic data. A workload generator executes seven banking scenarios, built from seven operation types, against the live database at a controlled rate, concurrency, burstiness and account skew, using transactions with row-level locking and ordered lock acquisition for transfers. A telemetry collector samples PostgreSQL statistics views and host resource usage once per second. A scikit-learn pipeline trains regression models that map pre-execution workload configuration to five measured outcomes, with safeguards against target leakage and warnings for out-of-range inputs. A what-if engine, a rule-based optimization advisor, a predicted-versus-actual validation engine, a REST API with background jobs and a React web interface complete the system.

All components are implemented and integrated. An experiment campaign of 240 workloads, with requested rates from 10 to 600 TPS, 2 to 64 workers and uniform or skewed account access, drove the database into lock contention and saturation without a single deadlock. Models trained on 247 runs were evaluated by cross-validation that holds out whole load settings, against a no-model baseline and against the measured variation between identical runs. On 16 further workloads that were predicted first and executed afterwards, the mean absolute error was 3.7 ms for average latency, 14.0 ms for P95 latency, 5.1 TPS for throughput and 3.8 percentage points for host CPU, in every case below the baseline. The models are accurate below about 350 TPS and weak above it, where the system saturates, and P95 latency is the least reliable target. The report also records two earlier evaluations that overstated accuracy and how validation runs exposed them.

Keywords: database performance, PostgreSQL, transaction processing, concurrency control, workload generation, telemetry, regression, digital twin, what-if analysis.

Table of Contents

Abstract 2
1. Introduction 5
1.1 Background and motivation 5
1.2 Problem statement 5
1.3 Objectives 5
1.4 Scope and status 6
1.5 Contributions 6
2. Background 7
2.1 Performance metrics 7
2.2 Transactions, locking and deadlock 7
2.3 PostgreSQL statistics views 7
2.4 Learning performance models 7
3. System Overview and Architecture 8
3.1 The experimental loop 8
3.2 Components 8
3.3 Technology choices 9
4. Database Design 10
4.1 Conceptual model 10
4.2 Relational schema and integrity constraints 11
4.3 Normalization 12
4.4 Indexes 12
4.5 Experiment platform tables 13
4.6 Synthetic data 13
5. Transaction Processing and Concurrency Control 13
5.1 The transfer transaction 13
5.2 ACID properties 14
5.3 Deadlock avoidance by lock ordering 14
5.4 Other operations 14
6. Workload Generation 15
6.1 Scenarios and operation mix 15
6.2 Configuration 16
6.3 Pacing and concurrency 16
6.4 What the latency measurement includes 16
6.5 Reproducibility 17
7. Telemetry Collection 17
7.1 Sampling 17
7.2 Experiment summary 18
7.3 Interpreting the telemetry 18
8. Machine Learning Pipeline 19
8.1 Dataset 19
8.2 Features and targets 19
8.3 Training and model selection 19
8.4 Prediction and out-of-range warnings 20
9. What-If Analysis 20
10. Optimization Advisor 21
11. Validation Framework 21
11.1 Lifecycle 21
11.2 Error metrics 22

12. API and User Interface....22
12.1 REST API....22
12.2 Web interface....23
13. Testing and Results....25
13.1 Automated tests....25
13.2 The experiment campaign....26
13.3 Telemetry for one experiment....27
13.4 Model training....28
13.5 What-if analysis....29
13.6 Optimization assessment....29
13.7 Predicted versus actual....30
13.8 Model study....30
14. Discussion....32
14.1 What has been achieved....32
14.2 How the first models failed, and what changed....32
14.3 Value of the result....33
15. Limitations....33
15.1 Data and models....33
15.2 Workload realism....33
15.3 Measurement....33
15.4 Engineering....34
16. Future Work....34
17. Conclusion....34
References....35
Appendix A. Reproducing the Experiments....36

1. Introduction

1.1 Background and motivation

An online transaction processing (OLTP) database does not have a single performance figure. Latency and throughput depend on how many clients are active, how quickly requests arrive and which operations they perform. Write transactions that update the same rows contend for locks, read queries compete for buffer cache and CPU, and the response time seen by a client is a result of all of these interactions [1]. A bank is a clear example: an ordinary weekday, a salary-credit day and a festival shopping peak place very different demands on the same schema.

The usual way to answer a capacity question is to run a load test for the configuration in question and measure the result, as standard benchmarks such as TPC-C [5] and frameworks such as OLTP-Bench [4] do. This is accurate but slow, and the number of possible configurations grows quickly with every parameter. A complementary approach is to treat a set of measured experiments as training data and to learn a model of the system's performance, in the spirit of a digital twin [2] and of machine-learning-based database tuning systems such as OtterTune [3]. FinTwin explores this approach at the scale of a course project.

1.2 Problem statement

Given a PostgreSQL database with a banking schema, and a workload described by a scenario, an operation mix, a target transaction rate, a concurrency level and a duration, estimate the latency, throughput, host CPU utilisation and lock waiting that the workload will produce, without executing it; and provide a way to check any such estimate against a real execution of the same workload.

The project deliberately models the performance of the database and not the business logic of a bank. The banking domain supplies a realistic transactional schema and realistic operations; it is not connected to any real financial system and uses only synthetic data.

1.3 Objectives

- Design a normalized relational schema for a simplified bank in PostgreSQL, with integrity enforced by keys and constraints.
- Implement atomic, isolated banking operations and a workload generator that executes configurable scenarios against the live database.
- Collect database and host telemetry for every experiment and store it alongside the workload results.
- Train regression models that predict performance from workload configuration only, and expose them through a prediction interface.
- Provide what-if comparison of workload configurations using the models.
- Detect likely bottlenecks in measured experiments with transparent, rule-based checks.
- Validate predictions by executing the predicted configuration and computing the error.

- Provide a REST API and a web interface that present only measured or model-derived values.

1.4 Scope and status

Table 1 summarises the status of each objective at the time of this review. The statements in this report were checked against the source code, the stored model artefacts and the recorded result files in the repository; where a capability is partial, the table says so.

Table 1. Status of project objectives.

Objective	Status	Evidence in the repository
Relational banking schema	Implemented	`database/schema.sql; 6 banking tables, 5 platform tables`
Transactions and workload generation	Implemented	`services/transfer_service.py, workload/runner.py; 7 scenarios, 7 operation types, burstiness and account skew`
Telemetry collection	Implemented	`telemetry/collector.py; tables telemetry_samples, experiment_summaries`
Performance prediction	Implemented; better than a no-model baseline on every target, weak above 350 TPS	`ml/ package; 5 persisted models trained on 247 runs`
What-if analysis	Implemented	`what_if.py; comparison and sweeps through API, interface and CLI`
Optimization advisor	Implemented	`optimization.py; six threshold rules, advisory only`
Predicted-versus-actual validation	Implemented; 16 checks of the current models	`validation.py; artifacts/m3-validation.json`
API and web interface	Implemented	`main.py (19 REST endpoints); web/ (eight pages)`

1.5 Contributions

- An end-to-end experimental platform in which workload execution, telemetry, model training, prediction and validation share one database and one experiment identifier, so that every prediction can be traced to the measurement that tests it.
- A banking workload generator with rate pacing that is separated from latency measurement, configurable burstiness and account skew, and deterministic operation selection from a seed.
- A leakage-safe learning pipeline: only quantities known before execution are admitted as features, evaluation holds out whole load settings, every model is compared with a no-model baseline and with run-to-run variation, training is refused below a minimum dataset size, and predictions carry out-of-range warnings.
- A validation workflow that records failed predictions and failed runs as failures instead of converting them into accuracy figures.

- A measured campaign of 240 workloads that reaches lock contention and saturation, and a study of how model error depends on the amount of data, the load region and the design of the experiments.
- An honest evaluation: the report shows where the models work, where they do not, and two occasions on which the project's own evaluation was too generous.

2. Background

2.1 Performance metrics

Throughput is the number of operations completed per unit time, reported here in transactions per second (TPS). Latency is the time one operation takes. Because latency distributions in database systems are skewed, the mean hides the slow requests that users notice, and percentiles are reported as well: P50 (median), P95 and P99. FinTwin computes percentiles by linear interpolation between order statistics over the per-operation latency samples of an experiment.

2.2 Transactions, locking and deadlock

A transaction is a unit of work that is atomic, consistent, isolated and durable (ACID) [1]. PostgreSQL implements isolation with multi-version concurrency control and row-level locks; `SELECT ... FOR UPDATE` locks the selected rows against concurrent updates until the transaction ends [7]. Two transactions that each hold a lock the other needs form a deadlock. One of the four necessary conditions for deadlock is circular wait, and a classical way to prevent it is to make every transaction acquire resources in the same global order [6]. FinTwin applies this technique to account rows.

2.3 PostgreSQL statistics views

PostgreSQL exposes its internal state through system views [7]. FinTwin reads `pg_stat_activity` (one row per server process, with its state and wait event), `pg_stat_database` (cumulative counters per database, including commits, rollbacks, blocks read and hit, and deadlocks) and `pg_locks` (locks held or awaited). The optional `pg_stat_statements` extension adds per-statement statistics but must be preloaded by the server; it is not enabled in the project's Docker configuration.

2.4 Learning performance models

Predicting performance from configuration is a supervised regression problem. FinTwin evaluates three standard regressors from scikit-learn [8]: ordinary least-squares linear regression, a random forest [9] and gradient-boosted regression trees [10]. Two properties of these models matter for the results in Section 13. A linear model extrapolates along a straight line outside the range of its training data, whether or not the real system does. A tree ensemble cannot extrapolate at all: outside the training range its prediction is constant. A further risk in this setting is target leakage, where information that is only available after the outcome is known enters the features and inflates apparent accuracy [11]; the pipeline is designed to exclude it.

3. System Overview and Architecture

3.1 The experimental loop

FinTwin is organised around a loop of five activities. **Generate:** a workload configuration is turned into a paced stream of banking operations. **Observe:** each operation's latency and outcome are recorded, while a sampler records database and host statistics. **Learn:** completed experiments become rows of a training set, from which one model per target metric is fitted. **Predict:** a new configuration is passed to the models to obtain estimated metrics, with warnings when it lies outside the training data. **Validate:** the same configuration is executed, and the estimates are compared with the measurements.

3.2 Components

Figure 1 shows the components and their dependencies. The FastAPI application is a thin layer: it validates requests and delegates to the same backend modules that the command-line scripts call directly. PostgreSQL is both the system under test and the store for experiment results, which keeps predictions, measurements and validations linked by foreign keys. Table 2 lists each component with its location in the source tree.

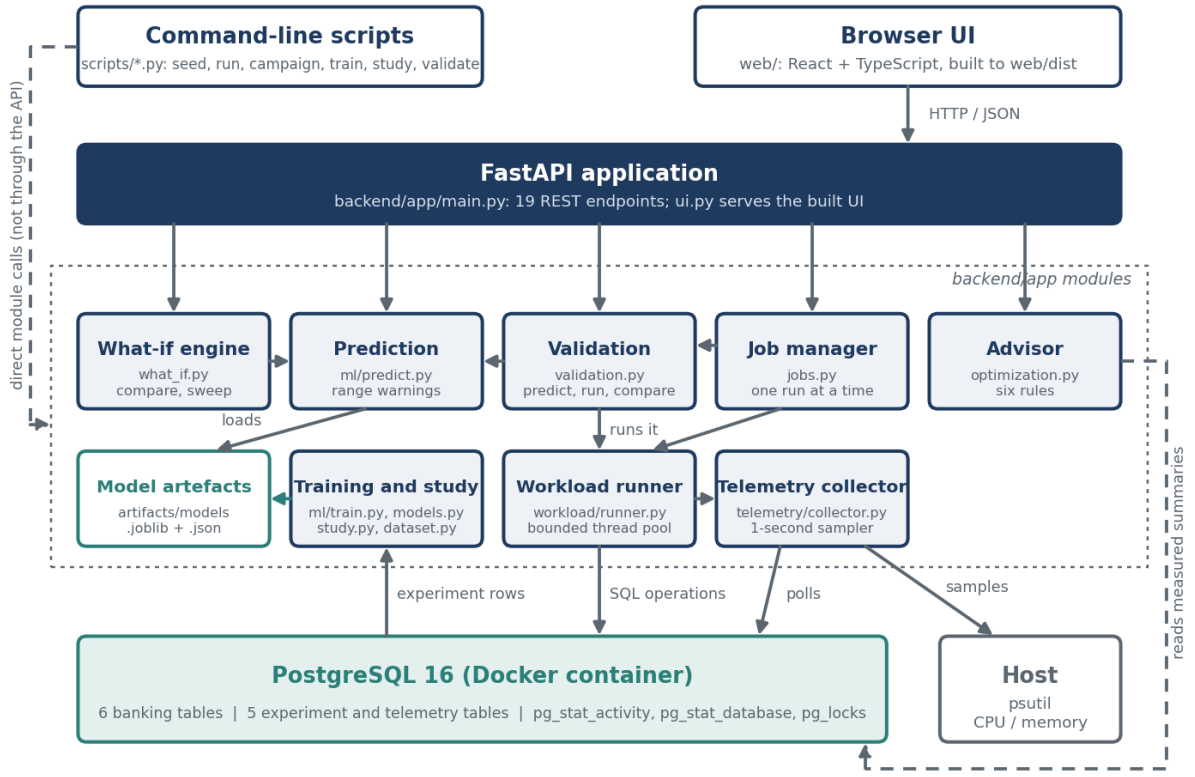


Figure 1. FinTwin architecture. Solid arrows are calls or data flow at run time; dashed arrows are paths that do not pass through the API.

Table 2. Components and their responsibilities.

Component	Source	Responsibility
Schema and seed	`database/schema.sql, scripts/seed_database.py`	Defines all tables, constraints and indexes; loads deterministic synthetic data (seed 42).
Transfer service	`backend/app/services/transfer_service.py`	Atomic fund transfer with ordered row locking.
Workload runner	`backend/app/workload/`	Scenario definitions, configuration validation, paced concurrent execution with burstiness and account skew, per-operation event logging.
Telemetry collector	`backend/app/telemetry/`	Background sampling of PostgreSQL views and host metrics; per-experiment summary.
ML pipeline	`backend/app/ml/`	Dataset extraction, feature definition, cross-validated training with model selection, prediction with range warnings, model study.
What-if engine	`backend/app/what_if.py`	Baseline-versus-modified comparison, parameter sweeps, sensitivity.
Optimization advisor	`backend/app/optimization.py`	Threshold rules over measured experiment metrics; advisory recommendations.
Validation engine	`backend/app/validation.py`	Predict, execute, compare; error metrics and aggregate summaries.
API	`backend/app/main.py, ui.py, jobs.py`	REST endpoints, background jobs, serving of the built interface.
Web interface	`web/`	Eight pages over the API, written in React and TypeScript and built to static files.

3.3 Technology choices

Table 3 lists the technologies used and the reason each was chosen.

Table 3. Technology stack and the reason for each choice.

Technology	Role	Reason
PostgreSQL 16 (Docker, postgres:16-alpine)	System under test and result store	ACID transactions, row-level locking and detailed statistics views; a container gives a reproducible server with a named volume and a health check.
Python 3.11+	Implementation language	Required for StrEnum; one language for workload, API and ML.
SQLAlchemy 2.x Core with psycopg 3	Database access	SQL is written explicitly with text(), so the statements that are measured are exactly the statements in the source; connection pooling is provided by the engine.
FastAPI and Pydantic	REST API	Declarative request validation (ranges, enumerations) and automatic conversion of errors to HTTP status codes [12].
pandas, scikit-learn, joblib	Dataset, models, persistence	Standard, interpretable regressors; a Pipeline stores preprocessing with the model.
psutil	Host metrics	Portable CPU and memory readings.
React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, Recharts	Web interface	Typed components and API client, cached server state and charts; the build output is static files served by the API.
pytest, httpx; Vitest, Testing Library	Testing	Backend unit and integration tests, including API tests through the FastAPI test client; component tests for the interface.

4. Database Design

4.1 Conceptual model

The banking domain is modelled with six entities. A **branch** serves many customers and holds many accounts. A **customer** owns one or more accounts. An **account** participates in many transactions, as the source, the destination or both roles of different transactions. A **merchant** receives card or point-of-sale payments. A **transaction** records one movement of money and references up to two accounts and, for payments, a merchant. A **loan** belongs to a customer, is linked to one of the customer's accounts and is managed by a branch. Figure 2 shows the resulting relational schema together with the five tables that the platform itself writes.

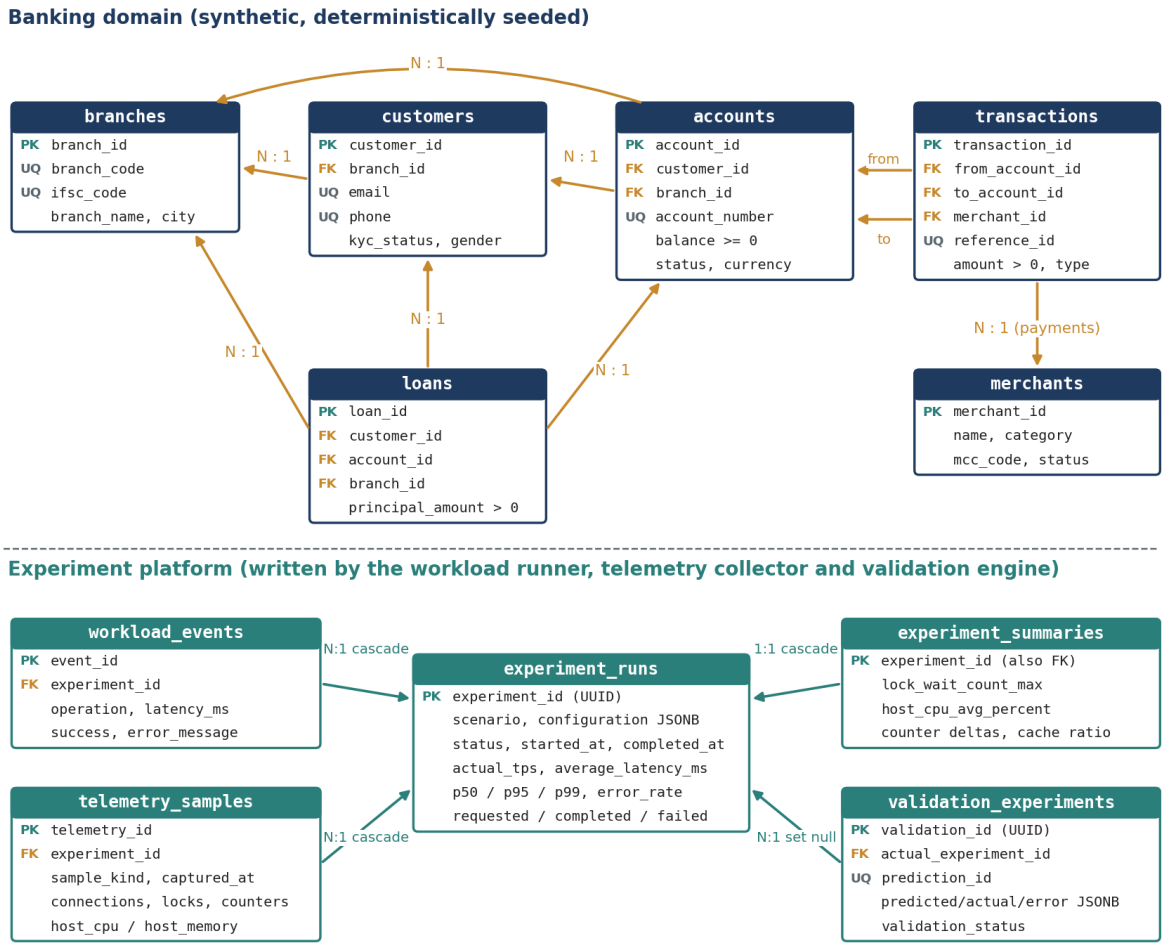


Figure 2. Relational schema defined in `database/schema.sql`. Arrows point from the referencing table to the referenced table. Only key and representative columns are shown.

4.2 Relational schema and integrity constraints

Every table has a single-column surrogate primary key (`BIGINT GENERATED BY DEFAULT AS IDENTITY` for banking tables, `UUID` for experiments and validations). Table 4 lists the keys and the main declarative constraints. Constraints are enforced by the database, so an invalid row is rejected regardless of which application code attempts to insert it.

Table 4. Banking tables: keys and principal constraints.

Table	Seeded rows	Foreign keys	Principal constraints
branches	100	none	branch_code and ifsc_code unique and non-empty
customers	1,000	branch_id	email and phone unique; gender and kyc_status restricted to enumerated values
accounts	1,500	customer_id, branch_id	account_number unique; balance >= 0; currency = 'INR'; a closed account must have a closed_date not earlier than open_date
merchants	100	none	status restricted to ACTIVE or INACTIVE
transactions	5,000	from_account_id, to_account_id, merchant_id	amount > 0; reference_id unique; source and destination must differ; shape constraint by type (below)
loans	200	customer_id, account_id, branch_id	principal_amount > 0; interest_rate >= 0; tenure_months > 0; end_date > start_date

The constraint transactions_shape_ck makes the meaning of each transaction type explicit in the schema: a TRANSFER must have both a source and a destination account, a DEPOSIT must have a destination, a WITHDRAWAL must have a source, and a PAYMENT must have a source account and a merchant. A structurally invalid transaction row therefore cannot be stored.

4.3 Normalization

All attributes are atomic and there are no repeating groups, so the tables are in first normal form. Every primary key is a single attribute, so no partial dependency on part of a key is possible and the tables are in second normal form. With respect to the functional dependencies that the design intends, in which each primary key determines the other attributes of its table, non-key attributes do not determine one another, and the design targets third normal form. Customer data is stored once and referenced from accounts; branch data is stored once and referenced from customers, accounts and loans; merchant data is stored once and referenced from payments.

Two points deserve an honest note because an examiner may raise them. First, loans stores both account_id and customer_id. If a loan's customer is always the owner of the linked account, then customer_id is determined by account_id, which is a transitive dependency; the schema keeps both columns for direct joins and does not enforce that they agree. Second, loans stores tenure_months together with start_date and end_date without a constraint relating them, and the seed data assigns them independently. Both are acceptable in a synthetic performance test bed but would be tightened in a production design.

4.4 Indexes

PostgreSQL creates an index for every primary key and unique constraint. The schema adds fifteen explicit indexes: nine on the banking tables (foreign keys of accounts, transactions and loans, plus transactions.transaction_time), two on workload_events (experiment and time), one composite index on telemetry_samples (experiment_id, captured_at)

for time-ordered retrieval of an experiment's samples, and three on `validation_experiments` (status, scenario and linked experiment). The indexes on `from_account_id` and `to_account_id` support the recent-transactions query of the workload.

4.5 Experiment platform tables

- `experiment_runs`: one row per workload execution, with the scenario, the full configuration as JSONB, status, operation counts, measured TPS and latency statistics.
- `workload_events`: one row per executed operation, with its type, latency, the time it waited for a worker, success flag and error message.
- `telemetry_samples`: timestamped snapshots of database and host metrics, marked INITIAL, SAMPLE or FINAL.
- `experiment_summaries`: one row per experiment with aggregated telemetry (maxima, counter deltas, averages) and the server's connection limit.
- `validation_experiments`: one row per validation, holding the configuration, predicted metrics, actual metrics, per-metric errors and a status.

Events, samples and summaries reference `experiment_runs` with `ON DELETE CASCADE`. A validation references its experiment with `ON DELETE SET NULL`, so a validation record survives deletion of the run it measured. The schema file is idempotent (`CREATE ... IF NOT EXISTS`) and is applied by `scripts/initialize_database.py`.

4.6 Synthetic data

`scripts/seed_database.py` truncates the six banking tables and reloads them inside one transaction, using a pseudo-random generator with a fixed seed of 42, so the same data is produced on every run. All seeded accounts are ACTIVE with balances between 500 and 199,999, and all seeded customers are KYC-verified. The seeded transaction history is independent of the seeded balances; it provides rows for read queries and is not a ledger that reconciles to the balances.

5. Transaction Processing and Concurrency Control

5.1 The transfer transaction

`transfer_funds(source, destination, amount)` is the central write operation. It rejects non-positive amounts and identical accounts before touching the database, and then performs the following steps inside a single transaction opened with `engine.begin()`:

- **Lock.** Both account rows are selected with `SELECT ... WHERE account_id IN (:first_id, :second_id) ORDER BY account_id FOR UPDATE`.
- **Validate.** Both accounts must exist and be ACTIVE, use the same currency, and the source balance must cover the amount. Any failure raises `TransferError`.
- **Update.** The source balance is decreased and the destination balance increased.
- **Record.** A TRANSFER row with a unique reference identifier is inserted into `transactions`.

- **Commit.** Leaving the `engine.begin()` block commits; an exception at any step rolls back every change.

5.2 ACID properties

Table 5 relates each ACID property to the mechanism that provides it in the transfer transaction.

Table 5. How the implementation provides each ACID property.

Property	Mechanism in FinTwin
Atomicity	The debit, the credit and the transaction record are executed in one database transaction; an exception rolls back all three. An integration test confirms that an insufficient-balance transfer leaves both balances unchanged.
Consistency	The application checks the balance before updating, and the database independently enforces `balance >= 0`, positive amounts and the transaction shape constraint.
Isolation	`FOR UPDATE` row locks prevent two concurrent transactions from reading and updating the same balance at once. The isolation level is PostgreSQL's default, READ COMMITTED; the code does not change it.
Durability	Provided by PostgreSQL's write-ahead log once the commit returns. The data directory is on a named Docker volume and survives container restarts.

5.3 Deadlock avoidance by lock ordering

Consider two simultaneous transfers, from account A to B and from B to A. If each transaction locked its source first, the first could hold A and wait for B while the second holds B and waits for A. FinTwin avoids this by requesting both rows in one statement ordered by `account_id`, so every transfer acquires the lower-numbered account first, whatever the direction of the transfer. With a single global acquisition order, a cycle of transactions waiting on account rows cannot form [6].

The other write operations of the workload (deposit, withdrawal and payment) each lock exactly one account row, so they cannot take part in a cycle over account rows either. The claim made here is limited to what the code establishes: ordered locking removes the circular-wait condition for account row locks in the implemented operations. No automated test runs opposing transfers concurrently. The experiment campaign provides empirical support: across 280 recorded runs and 383,385 operations, including runs in which up to 43 lock requests were waiting at once, PostgreSQL reported no deadlock (Section 13.2).

5.4 Other operations

Deposits, withdrawals and merchant payments follow the same pattern on a single account: lock the row with `FOR UPDATE`, check the balance where money leaves the account, update it, and insert the corresponding transaction row, all in one transaction. The two read operations (balance check and recent transactions) and the loan aggregation query run without explicit locks.

6. Workload Generation

6.1 Scenarios and operation mix

A scenario is a probability distribution over seven operation types. Seven scenarios are defined in `workload/config.py`; their default mixes are shown in Table 6 and Figure 3. A configuration may also supply its own mix, which must contain only positive weights that sum to one. Table 7 describes the database work performed by each operation type.

Table 6. Default operation mix of each scenario (percentage of operations).

Scenario	Balance check	Recent txns	Transfer	Deposit	Withdrawal	Payment	Loan aggr.
NORMAL_DAY	35	20	20	10	10	5	0
SALARY_DAY	25	10	20	35	0	10	0
MONTH_END	20	20	15	0	0	15	30
FESTIVAL_SPIKE	25	15	10	10	0	40	0
CARD_PAYMENT_SPIKE	20	15	5	0	0	60	0
LOAN_PROCESSING	20	5	10	10	0	0	55
HIGH_CONCURRENCY_ TRANSFER	15	10	70	0	0	5	0

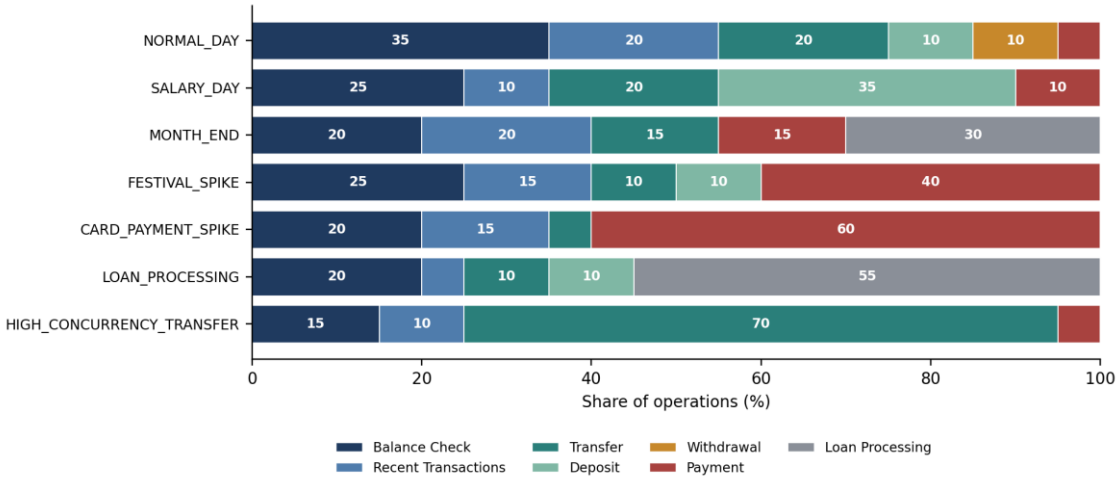


Figure 3. Operation mix of the seven scenarios, generated from `DEFAULT_MIXES` in the source.

Table 7. Operation types and the SQL work each performs.

Operation	Kind	Database work
BALANCE_CHECK	Read	Select the balance of one account by primary key.
RECENT_ TRANSACTIONS	Read	Select the 20 most recent transactions in which an account is source or destination.
TRANSFER	Write	Ordered lock of two accounts, two balance updates, one insert (Section 5.1).
DEPOSIT	Write	Lock one account, increase its balance, insert a DEPOSIT row.
WITHDRAWAL	Write	Lock one account, check and decrease its balance, insert a WITHDRAWAL row.

Operation	Kind	Database work
PAYMENT	Write	Choose a random active merchant, lock one account, check and decrease its balance, insert a PAYMENT row.
LOAN_PROCESSING	Read (aggregate)	Count loans and sum principal grouped by branch over the whole `loans` table.

6.2 Configuration

A workload is described by an immutable `WorkloadConfig`: scenario, concurrency (default 4, at most 64), target TPS (default 10), duration in seconds (default 10), an optional operation mix, a random seed (default 42), a burstiness value between 0 and 1 and an account skew of 0 or more. The constructor rejects out-of-range values and invalid mixes. Burstiness shapes arrivals (Section 6.3) and account skew decides which accounts are touched (Section 6.4). Both are stored with the experiment and used as model features. Every run in the campaign used burstiness 0, so burstiness is implemented and unit-tested but its effect has not been measured.

6.3 Pacing and concurrency

The runner computes the number of operations as the target TPS multiplied by the duration, computes a scheduled offset for each, and submits them to a thread pool whose size is the configured concurrency. With burstiness 0, operation i is scheduled at offset i divided by the target TPS. With burstiness b, each gap is (1 − b) times the mean gap plus b times an exponentially distributed gap with the same mean, so the average rate is unchanged while arrivals bunch together. The submitting thread sleeps until each offset is reached:

```
offsets = self._arrival_offsets(requested)
for index, target_offset in enumerate(offsets):
    remaining = target_offset - (perf_counter() - started_clock)
    if remaining > 0:

        time.sleep(remaining)
```

This is an open-loop schedule: arrivals do not wait for earlier operations to complete. The target TPS is therefore an input, whereas the actual TPS is a measurement, computed as completed operations divided by the wall-clock time until the last operation finishes. If the database cannot keep up, operations queue in the pool and the actual TPS falls below the target. The time each operation spends waiting for a free worker is recorded as its queue wait.

Each worker obtains connections from the SQLAlchemy engine's pool [13]. The pool is sized to the concurrency limit, with ten pooled connections and up to 64 overflow connections, so each of up to 64 workers can hold a connection at once, within PostgreSQL's configured limit of 100. The largest number of simultaneously active connections observed in a telemetry sample during the campaign was 48.

6.4 What the latency measurement includes

The latency of an operation is measured inside the worker thread with `time.perf_counter()`, from the moment the worker starts the operation until it finishes or raises an exception. Three consequences follow and should be kept in mind when reading the results:

- The pacing sleep and any time an operation spends waiting in the thread pool queue are **excluded**. Under overload, the reported latency is service time, not the response time an end user would see. The queue wait is stored per operation in `workload_events.queue_wait_ms`, so response time can be reconstructed: in the saturated run of Section 13.3 the mean queue wait was 2.3 s against a mean service time of 0.11 s.
- The identifiers of all active accounts are loaded once, before the run starts, so no lookup is **included** in the measured latency. With account skew 0 every account is equally likely. With skew s above 0 the account at rank r is chosen in proportion to 1 / r^s, a Zipf-like distribution that concentrates operations on a few hot accounts and so produces lock contention. Runs recorded before this change included a lookup query in every operation's latency; they are not used for training (Section 8.1).
- Writing the operation's row to `workload_events` happens after the timer stops and is **excluded**.

An operation counts as failed if it raises any exception, including a legitimate business rejection such as an insufficient balance; the message is stored with the event.

6.5 Reproducibility

The sequence of operation types is drawn from a generator seeded with the configuration's seed, and each operation's account and amount choices are drawn from a generator seeded with the seed plus the operation index. Two runs with the same configuration therefore issue the same logical operations in the same order. Timing, the interleaving of threads and the merchant chosen for a payment (selected in SQL with `ORDER BY RANDOM()`) are not deterministic.

7. Telemetry Collection

7.1 Sampling

When a run starts, the `TelemetryCollector` stores an INITIAL snapshot, then starts a daemon thread that stores a SAMPLE snapshot at a fixed interval (one second by default) until the run completes, and finally stores a FINAL snapshot. If a snapshot fails, the error is logged and counted in `telemetry_error_count`; the workload continues. Table 8 lists what one snapshot contains.

Table 8. Contents of one telemetry snapshot.

Source	Metrics	Notes
`pg_stat_activity`	Active, idle, waiting and lock-blocked sessions	Waiting means any wait event, including idle client waits; only sessions with wait type Lock are counted as blocked. Sessions of all databases on the server are counted.
`pg_stat_database`	Commits, rollbacks, blocks read and hit, tuples inserted, updated and deleted, deadlocks	Cumulative counters for the current database.
`pg_locks`	Total locks; locks not yet granted	The count of ungranted locks is the lock-wait metric.
`pg_database_size()`, `max_connections`	Database size in bytes; configured connection limit	
`psutil`	Host CPU percent; host memory percent	Whole-machine values, not PostgreSQL alone.

The cache hit ratio is derived from the block counters:

$$\text{cache hit ratio} = 100 \times \text{blks_hit} \,/\, (\text{blks_hit} + \text{blks_read})$$

7.2 Experiment summary

After the run, the collector writes one row to `experiment_summaries`: the error rate and latency percentiles from the workload result; the maximum of active connections, blocked sessions and ungranted locks over all snapshots; the differences between the FINAL and INITIAL commit, rollback and deadlock counters; the average host CPU (leaving out the INITIAL snapshot) and memory; the database size before and after; the configured connection limit; and the number of snapshots and errors.

7.3 Interpreting the telemetry

- **Host metrics are machine-wide.** CPU and memory include the workload generator, the API, Docker and every other process on the machine. A model trained on them is specific to the machine and to what else it was doing.
- **The first CPU reading is not meaningful.** `psutil.cpu_percent(None)` reports usage since the previous call and returns 0.0 on its first call [14]. The INITIAL snapshot is therefore left out of the CPU average.
- **The cache hit ratio is cumulative.** It is computed from counters accumulated since the server's statistics were last reset, taken at the FINAL snapshot, not from the blocks touched by the experiment alone.
- **Counter deltas include the platform's own activity.** Event inserts and telemetry inserts are commits too. The rollback delta is not a count of failed transactions: SQLAlchemy ends a connection block that was not committed with ROLLBACK, so every read-only operation contributes rollbacks. This is consistent with the recorded experiment in Section 13.3, which shows 1,152 rollbacks with an error rate of 0%.
- **Sampling is coarse.** With one-second snapshots, a lock wait that lasts a few milliseconds will usually fall between samples, so the lock-wait metric is a lower bound on contention.

8. Machine Learning Pipeline

8.1 Dataset

`ml/dataset.py` joins `experiment_runs` with `experiment_summaries` and expands the JSONB configuration into columns. Each experiment becomes one row. A configuration key is built from the scenario, concurrency, target TPS, duration, burstiness, account skew and the seven mix ratios; it identifies experiments that share a configuration regardless of seed. A load key, the pair of concurrency and target TPS, identifies runs made at the same load regardless of scenario. Three kinds of run are left out of the training set: runs that did not complete, runs recorded before the latency measurement was changed (Section 6.4), and runs whose elapsed time exceeds three times the requested duration, which indicates that the machine was suspended during the run.

8.2 Features and targets

Table 9 separates the model inputs, all of which are known before a workload runs, from the outputs, which are known only afterwards.

Table 9. Model inputs and outputs.

Role	Columns	Known when
Categorical feature	`scenario (one-hot encoded; unknown categories are ignored at prediction time)`	Before execution
Numeric features (12)	`concurrency, target_tps, duration_seconds, burstiness, account_skew, and seven operation-mix ratios`	Before execution
Targets (5)	`average_latency_ms, p95_latency_ms, actual_tps, host_cpu_avg_percent, lock_wait_count_max`	Only after execution

The preprocessing step is a scikit-learn `ColumnTransformer` that lists the feature columns explicitly and is created with `remainder="drop"`. Any other column of the dataset, including every measured quantity, is discarded before a model sees the data. Leakage of post-execution information into the features [11] is therefore prevented by construction, and a unit test asserts that no target column appears in the feature set.

8.3 Training and model selection

- **Gate.** Training is refused unless there are at least six usable runs, four distinct configurations, two scenarios and two load settings.
- **Folds.** The runs are divided into five folds with GroupKFold, grouped by load key, so that every run at a given concurrency and rate is in the same fold and each model is always scored at loads it was not trained on.
- **Candidates.** For each target and each fold, three pipelines are fitted: linear regression, a random forest of 100 trees and gradient boosting with default settings (random state 42 for both ensembles). Throughput is modelled as the delivered share of the requested rate and multiplied back, so that the prediction scales with a rate the model has not seen.
- **Selection and references.** The candidate with the lowest out-of-fold mean absolute error (MAE) is chosen; root mean squared error (RMSE) and the coefficient of determination

(R^2) are recorded as well. The same folds score a no-model baseline: the requested rate for throughput, and the scenario's mean in the training folds for the other targets. The mean standard deviation between runs of identical configuration is recorded as the noise floor.

- **Refit and persistence.** The chosen algorithm is refitted on every usable run and saved with joblib, next to a JSON file that records the algorithm, the cross-validated metrics of all three candidates, the baseline, the noise floor, the scenarios seen and the minimum and maximum of every numeric feature.

Grouping by load setting, and not only by configuration, matters. Section 14.2 describes how an earlier evaluation, grouped by configuration, reported low errors for models that could not interpolate between load levels.

8.4 Prediction and out-of-range warnings

`predict_performance(workload)` builds one feature row from a workload description, filling the mix ratios from the scenario's default mix when none is given, and applies each persisted model. It then compares the request with the recorded training metadata and returns a warning for a scenario that was not in the dataset and for each numeric feature that lies outside the recorded minimum and maximum, for example `target_tps=800 is outside the training range 10.0–600.0`. If no model artefacts exist, prediction fails with an explicit error instead of returning a default value; the API reports this as HTTP 409.

9. What-If Analysis

The what-if engine answers questions of the form “what does the model expect if concurrency is doubled?” without running a workload. `compare_workloads(baseline, modified)` normalizes both configurations, predicts each, and reports for every metric the baseline value, the modified value, the absolute change and the percentage change:

$$\text{percentage change} = 100 \times (\text{modified} - \text{baseline}) \,/\, |\text{baseline}|$$

The percentage change is undefined, and reported as null, when the baseline prediction is zero. The comparison is labelled HIGH_IMPACT if any metric changes by at least 20%, MEDIUM_IMPACT if any changes by at least 5%, and LOW_IMPACT otherwise. Warnings from both predictions are merged and returned with the result.

`sweep_workload` predicts a series of configurations that differ in one parameter (concurrency or target TPS), and `sensitivity_analysis` reduces a sweep to one number per metric: the change in the prediction between the last and the first point divided by the change in the parameter. The comparison and the sweep are available through the API and the web interface; `scripts/what_if.py` adds JSON and CSV export.

These outputs describe the trained model, not the database. A sensitivity value states how the model's prediction responds to a parameter; it is not evidence of a causal effect in PostgreSQL, and it is only as reliable as the model in the region being explored.

10. Optimization Advisor

The advisor inspects the measured metrics of one experiment and applies six threshold rules (Table 10). The thresholds are fields of a `Thresholds` dataclass and can be overridden. Each rule that fires produces a recommendation with an identifier, category, severity, description, the evidence values that triggered it, a suggested investigation, the expected benefit and a flag stating that the recommendation requires validation. Recommendations are sorted by severity. If no rule fires, the status is NORMAL.

Table 10. Rules of the optimization advisor.

Category	Condition	Severity
CPU	Average host CPU ≥ 80%	HIGH
MEMORY	Average host memory ≥ 85%	HIGH
CONCURRENCY	Maximum active connections ÷ `max_connections` ≥ 0.8	HIGH
LOCKING	Maximum count of ungranted locks > 0	HIGH
LATENCY	P95 latency ÷ average latency ≥ 2.0	MEDIUM
QUERY	Cache hit ratio < 95%	MEDIUM

The advisor is strictly advisory: it never alters PostgreSQL configuration, creates an index or rewrites a query. It also attempts to read the twenty most expensive statements from `pg_stat_statements`. When the extension is unavailable, as it is in the project's container, the query fails, the advisor continues with aggregate telemetry and attaches a warning that query-level analysis is unavailable. No query-level finding is produced without that evidence.

The connection rule needs `max_connections`. It is recorded in every telemetry sample and is now carried into `experiment_summaries`, so the rule is evaluated for stored experiments. In the run of Section 13.6 it did not fire: 48 active connections against a limit of 100 is a ratio of 0.48.

11. Validation Framework

11.1 Lifecycle

A validation tests one prediction against one real execution. Its lifecycle is shown in Figure 4. The record is created before anything else happens and moves through the states PENDING, RUNNING and COMPLETED or FAILED. Because the prediction is stored before the workload runs, it cannot be adjusted after the measurement is known. If prediction or execution raises an exception, the record is marked FAILED with the error message and no error metrics are computed.

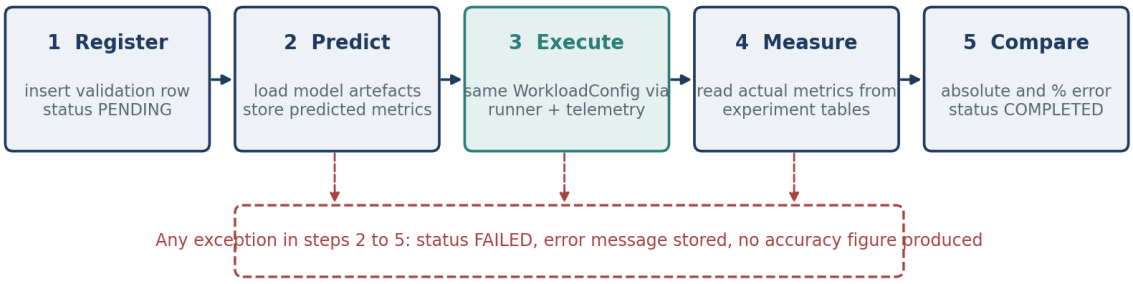


Figure 4. Lifecycle of a validation record as implemented in `run_validation`.

11.2 Error metrics

For a metric with predicted value $\hat{y}$ and measured value y, the per-run errors are:

$$\text{absolute error} = |\hat{y} - y| \qquad \text{percentage error} = 100 \times |\hat{y} - y| \,/\, |y|$$

The percentage error is null when the measured value is zero. Over a set of n completed validations, the summary reports, for each metric:

$$\text{MAE} = (1/n)\,\Sigma\,|\hat{y}_i - y_i| \qquad \text{RMSE} = \sqrt{(\,(1/n)\,\Sigma\,(\hat{y}_i - y_i)^2\,)}$$

$$R^2 = 1 - \Sigma\,(\hat{y}_i - y_i)^2 \,/\, \Sigma\,(y_i - \bar{y})^2 \qquad \text{MAPE} = (100/m)\,\Sigma\,|\hat{y}_i - y_i| \,/\, |y_i|$$

R^2 is null when the measured values have no variance or when every validation used the same configuration, and MAPE is taken over the m runs whose measured value is non-zero. Summaries are produced overall and grouped by scenario, by concurrency and by target TPS. A summary can be restricted to validations whose prediction was made after the current models were trained, so that checks of replaced models count neither for nor against the current ones. A separate script, `scripts/run_benchmark.py`, runs workloads with telemetry but without any model, providing measurements that do not depend on the ML pipeline.

12. API and User Interface

12.1 REST API

The FastAPI application (declared version 8.0.0) exposes nineteen endpoints under `/api`, listed in Table 11, and serves the built web interface from the same origin. Requests are validated by Pydantic models, which bound concurrency at 64, the rate at 1,000 TPS and the duration at 600 seconds. Database errors are returned as HTTP 503, unknown identifiers as 404, invalid workloads as 422 and missing model artefacts as 409. The API has no authentication; it is intended for local use.

Table 11. REST endpoints.

Method and path	Purpose
GET /api/health	Checks the database connection.
GET /api/database/summary	Row counts of the six banking tables.
GET /api/system/status	Database, telemetry and model availability.
GET /api/scenarios	The seven scenarios with their default mixes, and the input limits.
GET /api/experiments	Recent experiments with summary metrics (limit up to 500).
GET /api/experiments/{id}	One experiment with its telemetry samples and linked validations.
GET /api/experiments/{id}/latency	Latency and queue wait by operation type, and a latency histogram.
GET /api/recommendations/{id}	Optimization assessment of a stored experiment.
GET /api/models	For each model: algorithm, cross-validated error, baseline, noise floor and trained ranges.
POST /api/predictions	Predicted metrics and warnings for a workload.
POST /api/what-if	Baseline-versus-modified comparison.
POST /api/what-if/sweep	Predictions for a series of values of concurrency or rate, with sensitivity.
POST /api/jobs	Starts a workload or a validation in the background; returns a job identifier.
GET /api/jobs	Lists background jobs.
GET /api/jobs/{id}	State and result of one job.
GET /api/validation	List of validation records, optionally only those of the current models.
GET /api/validation/summary	Aggregate error metrics over completed validations, with the same option.
GET /api/validation/{id}	One validation record.
POST /api/validation	Runs a validation synchronously: predicts, executes the workload, compares.

Long-running work goes through `POST /api/jobs`, which accepts a workload or a validation, returns at once with a job identifier and runs the job in the background; `GET /api/jobs/{id}` reports its state. Jobs run one at a time so that two measurements never overlap. The job list is held in memory and is lost when the server restarts, although the experiments that the jobs produced are not. The synchronous `POST /api/validation` is retained for scripts.

12.2 Web interface

The interface is a single-page application in `web/`, written in React and TypeScript and built with Vite into static files that the API serves. It has eight pages: Overview (the latest run stated in a sentence, its measured figures, every run plotted as requested against delivered rate, and the latest check of a prediction), Experiments (all runs), a page for each run (telemetry over time, latency by operation, the advisor's findings and, where one exists, the prediction beside the measurement), Run (start a workload), Predict, What if (sweep one setting), Validation (predict, run, compare) and Models (each model's error beside the no-model baseline and the noise floor, and the ranges the models were trained on). Charts are

drawn with Recharts. A dark and a light theme are available. Figures 5 and 6 show two of the pages.

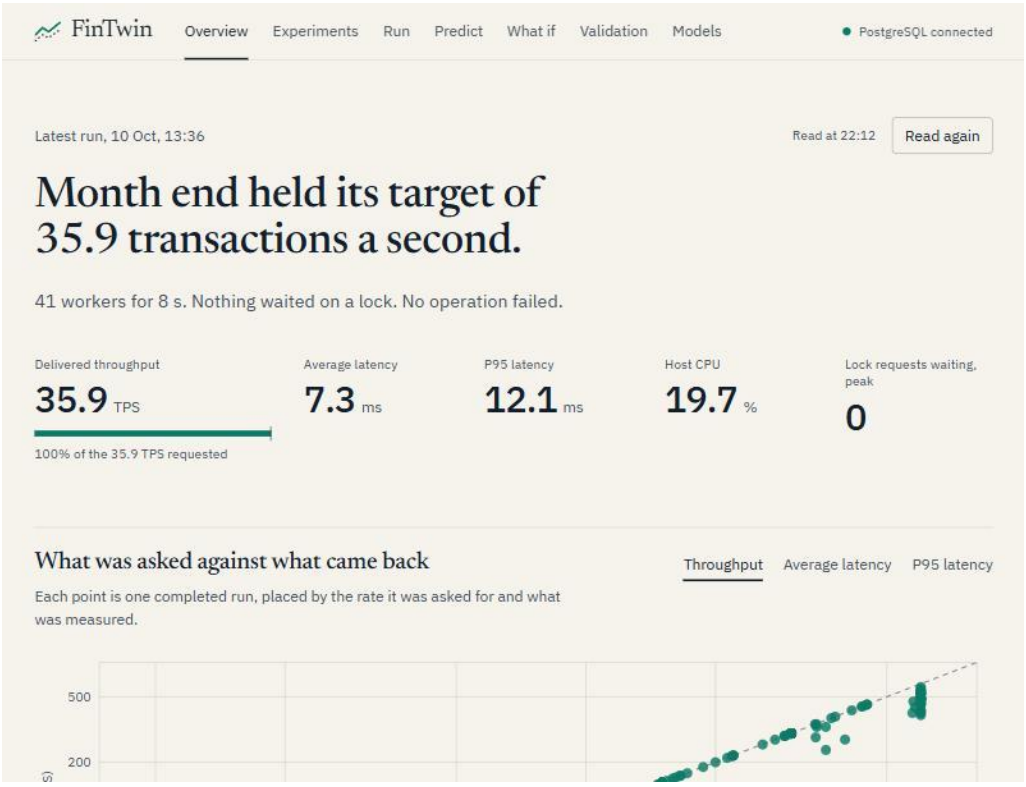


Figure 5. Overview page in the light theme: the latest run stated as a sentence, with its measured figures.

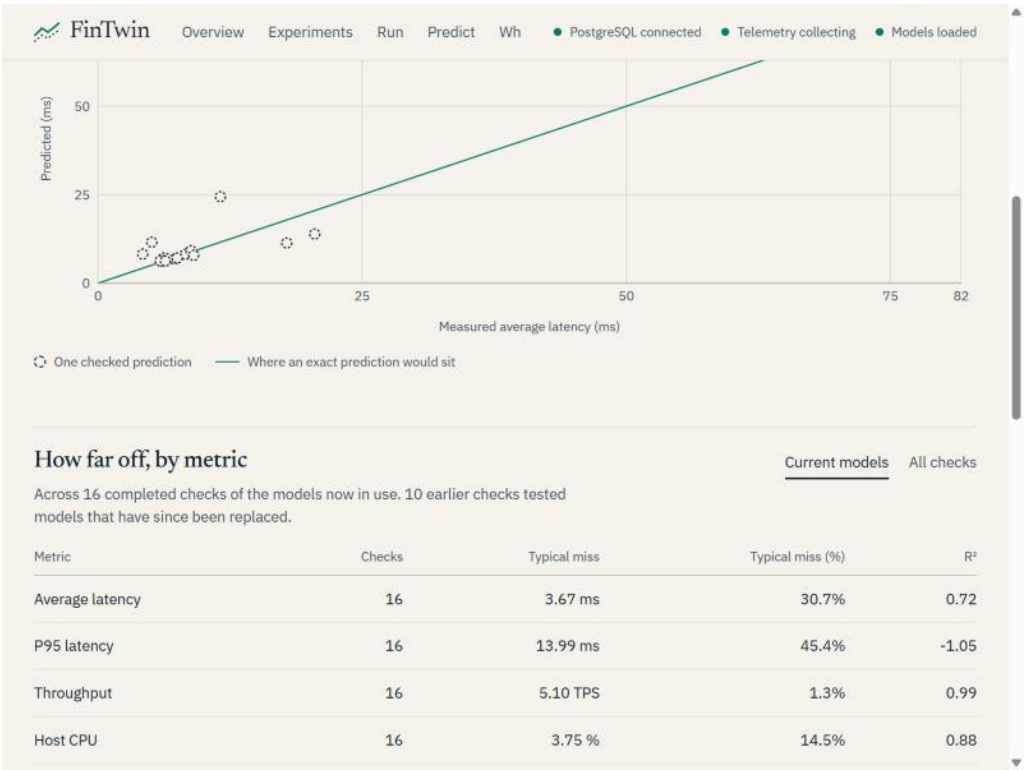


Figure 6. Validation page: predicted against measured for the checks of the current models, and the error by metric.

One visual rule runs through the interface: a measured value is drawn filled in a single signal colour and a predicted value is drawn hollow and dashed, so the two cannot be confused. The interface never displays a value the API did not return. Loading states are shown as placeholders, empty and error states are stated explicitly with the API's message, and missing numbers are shown as a dash and not as zero.

13. Testing and Results

This section separates three kinds of evidence: automated tests, measurements recorded from real PostgreSQL runs, and model outputs. The figures are taken from the project database and from result files written by the scripts (`artifacts/campaign.json`, `campaign-random.json`, `m3-validation.json`, `study.json` and `artifacts/models/*.json`). The measurements come from one development machine and from runs of eight seconds; they characterise this machine under these workloads and are not a general statement about PostgreSQL.

13.1 Automated tests

Table 12 lists the test modules, the number of test cases in each, and what they verify.

Table 12. Test modules and what each verifies.

Module	Cases	Needs PostgreSQL	What is verified
`test_phase1.py`	4	Yes	Tables exist and seed counts match; a successful transfer debits, credits and records one row; an insufficient-balance transfer changes nothing; health and summary endpoints respond.
`test_workload.py`	5	No	Required scenarios exist; default mix and seed; mixes that do not sum to one are rejected; non-positive concurrency, rate and duration are rejected.
`test_telemetry.py`	2	No	Percentile interpolation and average; counter delta with missing values.
`test_ml.py`	2	No	Feature set contains no target column; prediction without model artefacts raises an error.
`test_what_if.py`	4	No	Comparison deltas, sweep and sensitivity, input validation and warning propagation, using a stub predictor.
`test_optimization.py`	3	No	CPU and tail-latency rules fire; healthy metrics give no recommendation; lock rule carries its evidence.
`test_validation.py`	3	No	Absolute and percentage error; zero and missing actual values; summary by scenario.
`test_frontend.py`	1	No	The API serves the built interface and no longer the old one; unknown API paths return 404.
`test_m1_logic.py`	15	No	Even and bursty arrival offsets; uniform and skewed account choice; the concurrency limit and pool size; CPU average without the first sample; R^2 withheld for a single configuration; request bounds.
`test_m1_database.py`	4	Yes	Run detail and advisor when a summary is missing; a failed run is marked FAILED and its telemetry stops; the connection limit is stored.
`test_m2_logic.py`	14	No	Background jobs: result, failure, one at a time; latency breakdown and histogram; model registry; scenario, sweep and job endpoints.

Module	Cases	Needs PostgreSQL	What is verified
test_m2_database.py	4	Yes	Queue wait recorded for every operation; latency endpoint; a workload job run end to end.
test_m3_ml.py	13	No	Selection of training runs; baselines and noise floor; folds that hold out load settings; throughput as a share of the requested rate; campaign plans; interrupted runs.
test_m4_study.py	5	No	Learning curve, error by load band, transfer between experiment designs, noise floor.
test_f4.py	8	No	Validation summaries restricted to the current models; serving of the built interface and its limits.

The backend suite contains 87 test cases in fifteen modules, twelve of which need PostgreSQL. It was run on 10 October 2026 with Python 3.11 and the database available, and all cases passed. The interface has a further 73 component and logic tests in fifteen files, run with Vitest; all passed.

The tests cover the pure logic of each module, the transfer transaction, the failure path of the runner and the job manager. They do not run opposing transfers concurrently, and they do not test the timing of the telemetry sampler under load.

13.2 The experiment campaign

Two campaigns were run with scripts/run_campaign.py. The grid campaign executed every combination of seven scenarios, five load levels and two account skews (0 and 1.2) twice: 140 runs of eight seconds, in shuffled order so that drift in the machine does not line up with one scenario or one load. The random campaign executed 100 further runs with the number of workers drawn log-uniformly from 2 to 64, the requested rate from 10 to 600 TPS and the skew from 0, 0.6 and 1.2. No run failed. During the random campaign the machine was suspended twice; the two runs in progress recorded elapsed times of 31 minutes and 8 hours and are excluded from all analysis. Table 13 and Figure 7 summarise the grid campaign by load level.

Table 13. Grid campaign by load level: means over 28 runs (seven scenarios, two skews, two repetitions).

Workers	Requested TPS	Delivered TPS	Average latency (ms)	P95 latency (ms)	Runs with a lock wait
4	20	20.1	12.3	38.5	0 of 28
8	60	60.0	7.6	11.9	0 of 28
16	150	149.8	6.2	10.1	1 of 28
32	300	299.3	6.9	12.4	4 of 28
48	600	488.3	67.0	188.0	21 of 28

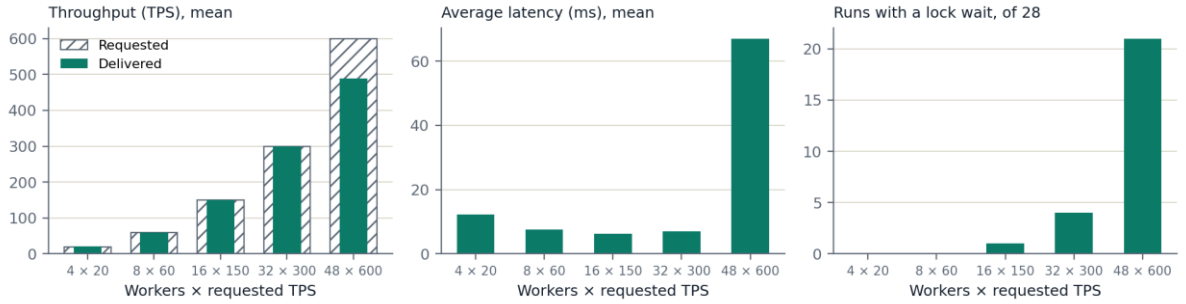


Figure 7. Grid campaign: requested and delivered throughput, average latency, and the number of runs with a lock wait, by load level.

Up to 300 TPS the database delivered the requested rate with single-digit average latency. At 600 TPS with 48 workers it saturated: 24 of the 28 runs delivered less than 90% of the requested rate, the mean delivered rate was 488 TPS and average latency rose tenfold. Account skew is what turns saturation into lock contention. For HIGH_CONCURRENCY_TRANSFER at the top level, the peak number of waiting lock requests averaged 1.5 with uniform account choice and 43 with skew 1.2, and P95 latency rose from 118 ms to 702 ms, while delivered throughput was almost the same (407 and 397 TPS).

In total the database holds 280 completed runs comprising 383,385 operations. 2,176 operations (0.6%) failed, every one of them a legitimate rejection for insufficient balance. PostgreSQL's deadlock counter did not increase in any run. The lightest load level is slower than the next two; the cause was not investigated.

13.3 Telemetry for one experiment

Table 14 shows the summary of one saturated run: HIGH_CONCURRENCY_TRANSFER with 48 workers, 600 TPS requested and account skew 1.2. All 4,800 operations completed and none failed, but the database delivered 387 TPS. 14 snapshots were stored with no telemetry errors. In the worst sample 42 sessions were blocked on a lock and 43 lock requests were waiting; no deadlock occurred. The median latency was 11.6 ms while P95 was 734 ms: most operations were fast, and those that queued behind a hot account were very slow. Operations waited 2.3 s on average for a free worker, which is the queueing that an open-loop schedule produces when the offered rate exceeds capacity.

Table 14. Telemetry summary of one saturated HIGH_CONCURRENCY_TRANSFER run (experiment a6dc595b).

Metric	Value	Metric	Value
Actual TPS	386.92	Commit delta	18,169
Average latency	114.04 ms	Rollback delta	1,152
P50 latency	11.60 ms	Deadlock delta	0
P95 latency	733.85 ms	Cache hit ratio	99.99%
P99 latency	1,571.51 ms	Average host CPU	53.63%
Error rate	0%	Average host memory	85.24%
Max. active connections	48	Max. blocked sessions	42
Max. ungranted locks	43	Mean queue wait	2,300 ms

13.4 Model training

The models were trained on 247 runs: the 140 grid runs, the 98 usable random-load runs and 9 earlier runs made with the current latency measurement. They cover 176 configurations and 109 load settings; 32 runs recorded lock waits and 29 were saturated. Table 15 gives, for each target, the chosen algorithm, its cross-validated MAE with whole load settings held out, the MAE of the no-model baseline on the same folds, and the noise floor.

Table 15. Chosen model per target: cross-validated error, no-model baseline and variation between identical runs (247 runs, 5 folds).

Target	Chosen model	MAE	No-model MAE	Between identical runs
Average latency (ms)	Random forest	3.42	15.22	1.01
P95 latency (ms)	Random forest	23.87	48.39	5.10
Throughput (TPS)	Random forest on rate share	6.63	15.45	3.32
Host CPU (%)	Linear regression	5.80	13.48	2.35
Max. lock waits	Random forest	2.04	4.39	0.38

Every model has less than half the error of predicting without a model, and none reaches the noise floor. The cross-validated R^2 values are 0.82 for average latency, 0.42 for P95 latency, 0.98 for throughput, 0.67 for host CPU and 0.52 for lock waits.

- **Average latency.** The model's error is about three times the noise floor. Most of it comes from the saturated region (Section 13.8).
- **P95 latency.** This is the weakest model. Tail latency under contention varies widely between otherwise similar runs, and an R^2 of 0.42 means that the model explains less than half of that variation.
- **Throughput.** Below saturation the delivered rate equals the requested rate, so the baseline is already nearly exact there. The model's advantage is that it predicts the shortfall of saturated runs.
- **Host CPU and lock waits.** Linear regression was chosen for CPU, which rises almost linearly with the requested rate; the measurement is machine-wide, so the model is

specific to this machine. The lock-wait model has learned that waits appear with skew at high load, but its error in that region remains close to the baseline's.

13.5 What-if analysis

The what-if engine was used to sweep the requested rate from 50 to 600 TPS at 32 workers for two workloads: NORMAL_DAY with uniform account choice, and HIGH_CONCURRENCY_TRANSFER with account skew 1.2. Every point lies inside the training range, so no warning was attached. Figure 8 shows the sweeps, and Table 16 compares 300 and 600 TPS for the second workload.

Table 16. What-if comparison: HIGH_CONCURRENCY_TRANSFER, 32 workers, skew 1.2, from 300 to 600 TPS (model predictions).

Predicted metric	At 300 TPS	At 600 TPS	Change
Average latency (ms)	8.8	107.2	+1,123%
P95 latency (ms)	16.3	635.5	+3,787%
Throughput (TPS)	298.8	403.6	+35%
Host CPU (%)	37.2	58.7	+58%
Max. lock waits	1.3	39.9	+2,867%

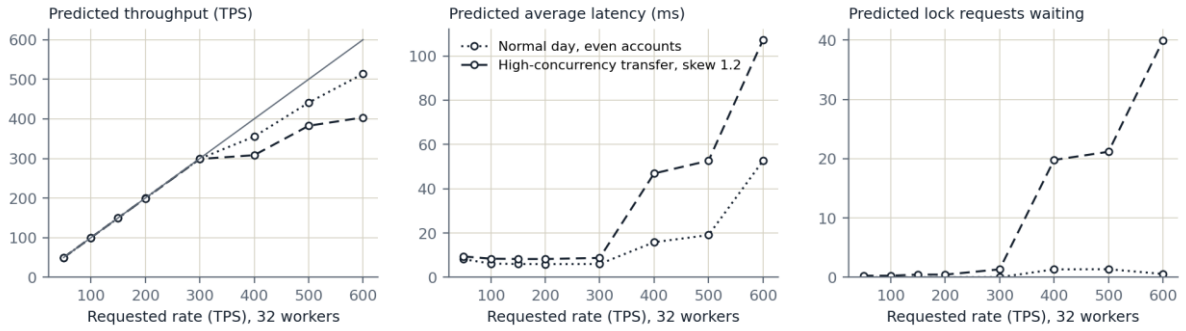


Figure 8. Predicted throughput, average latency and peak lock waits for a sweep of the requested rate at 32 workers. The grey line in the first panel marks delivered equal to requested.

The models predict that both workloads follow the requested rate up to about 300 TPS and fall behind it above that, and that the skewed transfer workload saturates earlier and far more sharply, with latency and lock waits rising together. This agrees with what the campaign measured at its top load level (Table 13). The shape between 300 and 600 TPS should be read with caution: it is the region where the models are least accurate (Section 13.8), and the steps in the curves are a property of tree models, not a statement about PostgreSQL. The comparison is labelled HIGH_IMPACT because a metric changes by more than 20%.

13.6 Optimization assessment

The advisor was run on the saturated run of Table 14. It returned status ELEVATED with three findings. Lock contention (HIGH): the number of waiting lock requests reached 43. Tail latency (MEDIUM): P95 was 6.4 times the average, against a threshold of 2. Memory pressure (HIGH): host memory averaged 85.2% against the 85% threshold, which is a property of the development machine and not of PostgreSQL and illustrates the limitation of

host-level metrics. The CPU, cache and connection rules did not fire. The advisor reported that query-level analysis was unavailable because pg_stat_statements could not be read.

13.7 Predicted versus actual

After the models were trained, 16 workloads were validated: each was predicted, the prediction was stored, and the workload was then executed. Twelve were new random draws from the ranges of the random campaign with a different seed, and four were loads between the grid levels on which an earlier generation of models had failed (Section 14.2). None of these runs was in the training set. Table 17 and Figure 9 compare the predictions with the measurements.

Table 17. 16 validations of the current models: mean absolute error of the model and of the no-model baseline.

Metric	Checks	Model MAE	No-model MAE	Model closer in
Average latency (ms)	16	3.67	9.54	13 of 16
P95 latency (ms)	16	13.99	24.40	12 of 16
Throughput (TPS)	16	5.10	9.71	9 of 16
Host CPU (%)	16	3.75	10.46	14 of 16
Max. lock waits	16	0.97	2.19	14 of 16

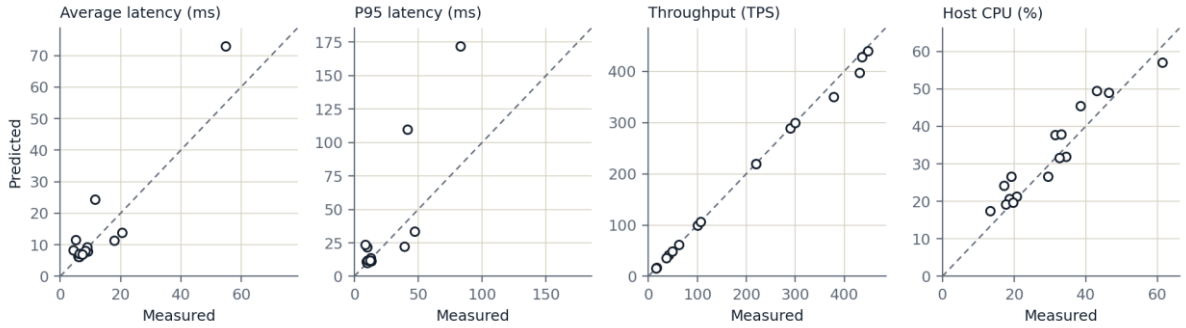


Figure 9. Predicted against measured for the 16 validations. The dashed line is an exact prediction.

The model was closer than the baseline on every metric on average, and the errors agree with the cross-validated estimates of Table 15 (3.67 against 3.42 ms for average latency, 5.10 against 6.63 TPS for throughput), which is evidence that the cross-validation is not optimistic. Two qualifications apply. For throughput the model was closer in only 9 of the 16 checks, because below saturation the requested rate is already an almost exact prediction; the model's lower mean error comes from the runs near saturation. And the largest errors are all at high load: for MONTH_END with 40 workers at 450 TPS the model predicted an average latency of 11.6 ms and 5.1 ms was measured, and for LOAN_PROCESSING with 22 workers at 379.6 TPS it predicted 350 TPS and 379 TPS was delivered. Sixteen checks are enough to confirm the size of the error, not to map it.

13.8 Model study

`scripts/run_study.py` asks four further questions of the 247 training runs; its output is `artifacts/study.json`. **How much data is needed?** Figure 10 shows the held-out error

after training on random subsets of the runs. The error falls quickly up to about 100 runs and little after that: from 100 to 198 runs the average-latency error falls from 4.2 to 3.4 ms and the P95 error from 25.3 to 24.0 ms. More runs of the same kind would not close the gap to the noise floor.

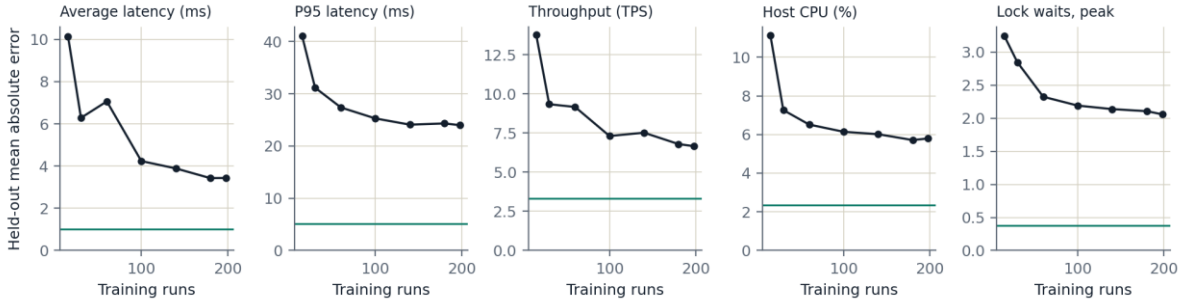


Figure 10. Learning curves: held-out mean absolute error against the number of training runs, with whole load settings held out. The horizontal line is the variation between identical runs.

Where is the error? Figure 11 splits the held-out error by requested rate. Below 350 TPS (210 runs) the average-latency error is at most 2.3 ms and the throughput error at most 1.8 TPS. In the 37 runs at 350 TPS and above, where the system saturates, the errors are 15.0 ms and 40.3 TPS. That is still well below the baseline for average latency and throughput, but for P95 latency (130 against 155 ms) and lock waits (12.6 against 14.6) the model is barely better than no model. The models are reliable in the unsaturated region and indicative only in the saturated one.

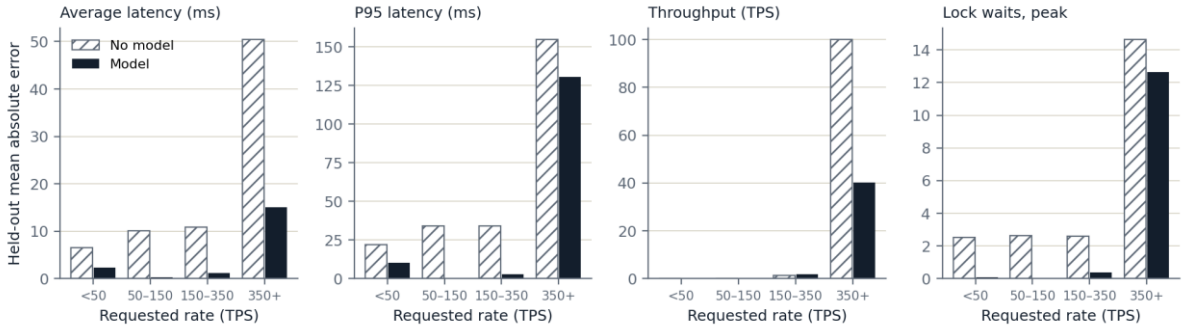


Figure 11. Held-out error by requested rate: the model against the no-model baseline.

How large is the noise? Among the 71 configurations that were run more than once, the standard deviation between identical runs averaged 6.4% of the mean for average latency, 11.1% for P95 latency, 9.0% for host CPU and 0.7% for throughput. No model of the configuration alone can be more accurate than this.

Does the design of the experiments matter? Table 18 fits each model on the runs of one campaign and scores it on the other. Models fitted on the 98 random-load runs predict the grid runs better than the baseline on every target. Models fitted on the 140 grid runs, which contain only five distinct loads, also do, but for throughput only because it is modelled as a share of the requested rate: the same random forest fitted on raw throughput has an error of 74.0 TPS on the random-load runs, against 4.8 TPS for the share model and 5.4 TPS for the baseline. A tree model given five load levels can only answer with one of those five.

Table 18. Mean absolute error when a model is fitted on one campaign and scored on the other, beside the no-model baseline.

Metric	Grid → random: model	No model	Random → grid: model	No model
Average latency (ms)	6.40	12.18	3.61	14.06
P95 latency (ms)	21.20	40.39	31.22	43.46
Throughput (TPS)	4.79	5.44	8.52	22.54
Host CPU (%)	7.74	13.17	7.45	12.92
Max. lock waits	0.61	3.77	3.20	3.78

14. Discussion

14.1 What has been achieved

The engineering objectives have been met. The schema enforces its invariants declaratively; the transfer transaction is atomic and uses ordered locking; the workload generator executes reproducible operation sequences at a controlled rate; telemetry is captured and linked to each run; the learning pipeline excludes leakage and refuses to train or predict without sufficient basis; what-if results carry range warnings; the advisor reports only what its evidence supports; and the validation engine ties each prediction to a measurement. The whole loop runs end to end from the command line, the API and the browser. The measurement objectives have now been met as well: the workload reaches contention and saturation, and the models are judged against a baseline, a noise floor and fresh runs.

14.2 How the first models failed, and what changed

The models in the first version of this report were trained on seven ten-second runs at 10 to 30 TPS. Their single validation run showed errors of 49.7% for average latency, 66.5% for P95 latency, 20.6% for throughput and 50.5% for host CPU. Three causes were identified, and each has been addressed.

The dataset was too small and did not stress the database. With an operation taking about 10 ms and at most 30 TPS, fewer than one operation was in progress at a time, so lock waits and saturation never appeared. The campaign of Section 13.2 raises the load to 600 TPS with up to 64 workers, and account skew makes contention reachable.

The measurement included work that was not the operation. Every operation's latency included a lookup of account identifiers, and operations touched only the first 1,000 accounts. The lookup now happens once per run, all active accounts are eligible, and the time spent waiting for a worker is recorded separately.

The evaluation was too weak to catch a bad model, twice. The first models were scored on two held-out runs. The second generation, trained on the grid campaign and cross-validated with whole configurations held out, looked accurate: the throughput model's error was 4.1 TPS against a baseline of 22.6. Four validation runs at loads between the grid levels then showed throughput predictions 25% to 40% too low; for 100 TPS requested the model

predicted 60, the nearest grid level. Other scenarios at the same load had stayed in the training folds, so the evaluation never asked the models to interpolate. Three changes followed: folds now hold out whole load settings, throughput is modelled as a share of the requested rate, and a campaign with randomly drawn loads was added. Without the validation step the misleading figures would have been reported.

14.3 Value of the result

That episode shows the value of the validation framework. A platform that only ever confirmed its own predictions would be untrustworthy. This one contradicted its own cross-validation, the stored evidence was sufficient to diagnose why, and the corrected evaluation agrees with fresh measurements. The result is a set of models with stated accuracy and stated limits, not a claim that performance can be predicted in general.

15. Limitations

15.1 Data and models

- 247 training runs and 16 validations, all from one machine and all eight to ten seconds long. Longer runs may behave differently as checkpoints, autovacuum and cache effects come into play.
- Above about 350 TPS the models are weak: P95 latency and lock waits are predicted little better than by the baseline, and latency tends to be over-predicted for workloads that turn out not to saturate.
- Burstiness was 0 in every run, so predictions for bursty arrivals are extrapolations.
- Predictions have no confidence interval. The noise floor and the error by load band describe the models as a whole, not a single prediction.
- Models are specific to the machine on which the experiments were run.

15.2 Workload realism

- Account skew follows a fixed Zipf-like rule over the list of accounts. Real hot spots move over time.
- Burstiness is implemented but its effect has not been measured (Section 6.2).
- The banking tables are small, and the whole database (about 132 MB after the campaigns, most of it recorded events and transactions) fits in memory, so the cache hit ratio is always above 99%.
- Scenarios differ only in their operation mix; none varies the arrival rate over time within a run.

15.3 Measurement

- Latency is service time. Queue wait is recorded separately and is not a model target, so the models do not predict the response time a user would see under overload.
- CPU and memory are host-wide, and the cache hit ratio is cumulative (Section 7.3).
- Lock waits are sampled once per second and so undercount short waits.

- `pg_stat_statements` is not enabled, so there is no per-query evidence and no query-plan analysis.

15.4 Engineering

- Background jobs are held in memory and run one at a time; a server restart loses the job list, though not the experiments.
- A campaign must be run with the machine kept awake; two runs were lost to a suspension (Section 13.2).
- The API has no authentication and is intended for local use only.
- Only PostgreSQL is containerised; the application runs from a local Python environment and the interface needs a build step. There is no continuous integration.
- No automated test runs opposing transfers concurrently; the evidence against deadlock is empirical (Section 5.3).
- The interface was checked at phone and laptop widths in both themes, but not with a screen reader. The screenshots in this report were taken in a narrow window.

16. Future Work

- **Cover the saturated region.** Add runs between 300 and 600 TPS and beyond, with repetitions. That is where the error is concentrated, and more random-load runs elsewhere no longer help (Section 13.8).
- **Run longer and vary burstiness.** Use runs of minutes instead of seconds, and include bursty arrivals so that the burstiness feature is learned from data.
- **Improve measurement.** Measure the PostgreSQL container's CPU and memory instead of the host's; compute per-experiment cache statistics from counter deltas; enable pg_stat_statements; sample lock waits more often.
- **Report uncertainty.** Give each prediction an interval, for example from the spread of the forest's trees or from quantile regression, and predict response time including queue wait.
- **Extend the advisor.** Add evidence from query statistics and EXPLAIN (ANALYZE) once available.
- **Harden the application.** Persist jobs, add authentication, containerise the application, test opposing transfers under concurrency and add continuous integration.

17. Conclusion

FinTwin set out to build a system that can learn how a PostgreSQL banking database performs under different workloads and can check its own predictions. The platform that this requires has been built and integrated: a constrained relational schema, atomic and deadlock-aware transactions, a configurable workload generator, a telemetry collector, a leakage-safe learning pipeline, a what-if engine, a rule-based advisor, a validation engine, a REST API and a web interface, with automated tests for the logic of each part.

The predictive component has moved from a proof of mechanism to a measured result. Trained on 247 runs that include lock contention and saturation, the models beat a no-model

baseline on every target, both in cross-validation and on 16 workloads predicted before they were run, with a mean error of 3.7 ms in average latency and 5.1 TPS in throughput. They are reliable below about 350 TPS and only indicative above it, and tail latency remains the hardest target. The project's first two evaluations were both too generous, and it was the validation engine that showed this. The principal contribution is therefore the experimental platform and its discipline of keeping predictions and measurements separate and comparable, now demonstrated by a case in which that discipline changed the conclusion.

References

[1] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2019.

[2] M. Grieves and J. Vickers, “Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems,” in Transdisciplinary Perspectives on Complex Systems, Cham: Springer, 2017, pp. 85–113.

[3] D. Van Aken, A. Pavlo, G. J. Gordon and B. Zhang, “Automatic Database Management System Tuning Through Large-scale Machine Learning,” in Proc. ACM SIGMOD International Conference on Management of Data, 2017, pp. 1009–1024.

[4] D. E. Difallah, A. Pavlo, C. Curino and P. Cudré-Mauroux, “OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases,” Proceedings of the VLDB Endowment, vol. 7, no. 4, pp. 277–288, 2013.

[5] Transaction Processing Performance Council, TPC Benchmark C Standard Specification, Revision 5.11, 2010.

[6] E. G. Coffman, M. J. Elphick and A. Shoshani, “System Deadlocks,” ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, 1971.

[7] The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: “Monitoring Database Activity”, “Explicit Locking” and “pg_stat_statements”. Available: https://www.postgresql.org/docs/16/

[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[11] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, “Leakage in Data Mining: Formulation, Detection, and Avoidance,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, article 15, 2012.

[12] FastAPI documentation. Available: https://fastapi.tiangolo.com/

[13] SQLAlchemy 2.0 documentation, “Connection Pooling”. Available: https://docs.sqlalchemy.org/en/20/core/pooling.html

[14] psutil documentation, “psutil.cpu_percent”. Available: https://psutil.readthedocs.io/

Appendix A. Reproducing the Experiments

The commands below are for Windows PowerShell from the project root and assume Docker, Python 3.11 or later and Node.js.

```
Copy-Item .env.example .env
docker compose up -d
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
$env:PYTHONPATH = (Get-Location).Path
python scripts/initialize_database.py
python scripts/seed_database.py
```

Run a workload with telemetry, run the two campaigns (keep the machine awake), then train and study the models:

```
python scripts/run_workload.py --scenario NORMAL_DAY --concurrency 4 `
    --target-tps 10 --duration 10
python scripts/run_campaign.py --design grid --repetitions 2
python scripts/run_campaign.py --design random --runs 100 `

    --output artifacts/campaign-random.json
python scripts/train_models.py
python scripts/run_study.py
```

Predict, compare, assess and validate:

```
python scripts/predict_workload.py --scenario NORMAL_DAY --concurrency 16 --target-tps 100
python scripts/what_if.py --concurrency 32 --target-tps 300 --change-tps 600
python scripts/what_if.py --concurrency 32 --target-tps 100 `
    --sweep-tps 50 100 200 300 400 600
python scripts/analyze_performance.py --experiment-id <UUID>
python scripts/validate_prediction.py --scenario NORMAL_DAY --concurrency 12 `
    --target-tps 100
python scripts/run_benchmark.py --scenarios NORMAL_DAY SALARY_DAY `
    --format csv --output artifacts/benchmark.csv
```

Build the web interface, start the API and run the tests:

```
npm --prefix web install ; npm --prefix web run build
python -m uvicorn backend.app.main:app   # then open http://127.0.0.1:8000/

python -m pytest ; npm --prefix web test
```